GOOGLE CLOUD STORAGE COMPATIBILITY

Connect Your Google Cloud Storage Workloads to Inaya

A step-by-step guide for pilot IT/ops teams: use Inaya's real Google Cloud Storage-compatible XML API endpoint with your existing HMAC credentials, tools, and client libraries.

SECTION 01

What This Gives You

Inaya's storage endpoint accepts Google Cloud Storage's own documented XML API authentication conventions -- both GCS's native GOOG4-HMAC-SHA256 signing scheme and its AWS4-HMAC-SHA256 interoperability mode. If your existing code, scripts, or client libraries authenticate to Google Cloud Storage using an HMAC access key and secret, they can point at Inaya instead by changing only the endpoint and credential -- no protocol rewrite. Every byte is still encrypted, sharded, and redundantly stored across Inaya's decentralized storage network underneath.

ENDPOINT	ACCEPTED SIGNING MODES
https://<your-inaya-host>/api/s3	GOOG4-HMAC-SHA256 (Google's native scheme) and AWS4-HMAC-SHA256 (Google's own documented S3-interoperability mode)

What to put in place of <your-inaya-host>. Same as the Multi-Cloud Storage Compatibility guide: for the standard shared Inaya platform, use www.inayanetwork.com. If your pilot runs on a dedicated hostname, your Inaya account manager will provide it.

SECTION 02

Step 1 - Get Your Credentials

1. **1. Sign in to Business Workspace.** Open the "S3-Compatible Storage" panel from the left navigation -- the same panel used for AWS and Azure access.
2. **2. Click "+ New S3 credential."** One credential works across AWS, Azure, and Google Cloud Storage signing conventions -- you do not need a separate credential per cloud.
3. **3. (Optional) Restrict its scope.** Limit the credential to one bucket, one folder path, specific operations (READ/WRITE/DELETE/LIST), and/or an expiry date before creating it.
4. **4. Save the Access Key ID and Secret Access Key immediately.** Shown exactly once. Store it the way you would store any other cloud credential.

SECTION 03

Step 2 - Authenticate with Google's Native GOOG4 Scheme

If your existing code uses Google's official client libraries (for example, the Node.js @google-cloud/storage package or the Python google-cloud-storage package) configured with an HMAC key pair and a custom API endpoint, it produces a native GOOG4-HMAC-SHA256 signature. Point it at Inaya by overriding the endpoint and supplying your Inaya-issued HMAC credential in place of a Google one.

CONCEPTUAL EXAMPLE (CONSULT YOUR SPECIFIC CLIENT LIBRARY'S OWN DOCUMENTATION FOR THE EXACT ENDPOINT-OVERRIDE OPTION NAME)

```
endpoint: "https://<your-inaya-host>/api/s3"
hmacAccessId: "<your Access Key ID>"
hmacSecret: "<your Secret Access Key>"
```

> Inaya's server verifies the real, published GOOG4-HMAC-SHA256 algorithm exactly as Google documents it (canonical request, x-goog-date/x-goog-content-sha256 headers, the goog4_request credential scope) -- this is not a simplified approximation.

SECTION 04

Step 3 - Authenticate with the AWS4 Interoperability Mode

Google also documents an "S3 interoperability" signing mode for its own XML API -- genuine AWS4-HMAC-SHA256, byte-identical to real AWS SigV4. Any tool built to speak this mode (including the standard AWS CLI and AWS SDKs, pointed at a custom endpoint) already works against Inaya with zero changes.

TERMINAL -- USING THE AWS CLI

```
aws configure --profile inaya
# AWS Access Key ID:     <your Access Key ID>
# AWS Secret Access Key: <your Secret Access Key>
# Default region name:   inaya

aws s3 ls --profile inaya --endpoint-url https://<your-inaya-host>/api/s3
aws s3 cp report.pdf s3://pilot-bucket/ --profile inaya --endpoint-url https://<your-inaya-host>/api/s3
```

SECTION 05

Step 4 - Verify the Pilot Connection

1. **1. Create a test bucket.** Using whichever authentication path from Steps 2-3 matches your existing tooling.
2. **2. Upload a small test file, then list the bucket to confirm it appears.**
3. **3. Download it back and compare.** The downloaded file must be byte-for-byte identical to what you uploaded.
4. **4. Confirm it's visible in Business Workspace.** Confirms Inaya's real storage/encryption/redundancy pipeline received it, not just a local acknowledgment.
5. **5. Delete the test object and bucket.** Confirms delete permissions work end-to-end before moving real data.

SECTION 06

What's Tested, and One Honest Limitation

- Object create/read/list/delete, byte-range downloads, and metadata: tested and working via both signing modes above.
- Large-file uploads via Google's XML API multipart mechanism (the same shape as AWS S3 multipart): tested and working.
- Signing in with a Google account directly: a Google ID token is now a real third authentication path (alongside AWS4/GOOG4 HMAC signatures) -- it's verified for real against Google, never locally trusted, and mapped to an existing active Inaya org membership; an unmapped identity is rejected, not silently granted access.
- Temporary signed download links: a new ?presign endpoint issues a time-limited URL (capped at 7 days) that can never grant more than its creator's own credential already permits. 9/9 automated tests passing, including tamper detection on the object path, expiry, method and signature.
- Addressing a bucket via a subdomain (bucket.your-endpoint.com style) rather than a path: implemented as a pure routing rewrite, inert by default until an operator turns it on. 18/18 live checks passed against real DNS, including a real bug found and fixed (a Next.js routing quirk that could have let a virtual-hosted request's permission check silently disagree with the bucket it actually operated on).

gsutil's S3-compatible mode -- still untested, for a narrower reason than before. The original crash we found in Google's own gsutil tool (it rejected any endpoint address that didn't look like a standard Amazon address) is now confirmed avoidable through supported configuration alone. Retesting after that fix hits a separate, later-stage connection issue inside gsutil's own legacy connection-handling code -- still unresolved, and still not an Inaya-side gap. If your workflow depends specifically on gsutil, talk to your Inaya technical contact.

gcloud storage -- the modern Google Cloud CLI -- works. Distinct from the older gsutil tool above: gcloud storage's own S3-interoperability mode (which Google itself labels experimental) was tested against a real Inaya endpoint with real credentials. Upload, download and list all confirmed working, including a byte-identical download. Metadata and delete showed the same rough edges Google's own tooling discloses as unstable for this feature -- not an Inaya-side gap.

SECTION 07

Getting Help

During the pilot, route connection issues or questions about scope/permissions to your assigned Inaya technical contact rather than general support -- pilot feedback shapes the general-availability release.